



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs

Joseph Spracklen, Raveen Wijewickrama, and A H M Nazmus Sakib, *University of Texas at San Antonio*; Anindya Maiti, *University of Oklahoma*; Bimal Viswanath, *Virginia Tech*; Murtuza Jadliwala, *University of Texas at San Antonio*

<https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen>

**This paper is included in the Proceedings of the
34th USENIX Security Symposium.**

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the
34th USENIX Security Symposium is sponsored by USENIX.

We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs

Joseph Spracklen

University of Texas at San Antonio

Raveen Wijewickrama

University of Texas at San Antonio

A H M Nazmus Sakib

University of Texas at San Antonio

Anindya Maiti

University of Oklahoma

Bimal Viswanath

Virginia Tech

Murtuza Jadliwala

University of Texas at San Antonio

Abstract

The reliance of popular programming languages such as Python and JavaScript on centralized package repositories and open-source software, combined with the emergence of code-generating Large Language Models (LLMs), has created a new type of threat to the software supply chain: *package hallucinations*. These hallucinations, which arise from fact-conflicting errors when generating code using LLMs, represent a novel form of package confusion attack that poses a critical threat to the integrity of the software supply chain. This paper conducts a rigorous and comprehensive evaluation of package hallucinations across different programming languages, settings, and parameters, exploring how a diverse set of models and configurations affect the likelihood of generating erroneous package recommendations and identifying the root causes of this phenomenon. Using 16 popular LLMs for code generation and two unique prompt datasets, we generate 576,000 code samples in two programming languages that we analyze for package hallucinations. Our findings reveal that the average percentage of hallucinated packages is at least 5.2% for commercial models and 21.7% for open-source models, including a staggering 205,474 unique examples of hallucinated package names, further underscoring the severity and pervasiveness of this threat. To overcome this problem, we implement several hallucination mitigation strategies and show that they are able to significantly reduce the number of package hallucinations while maintaining code quality. Our experiments and findings highlight package hallucinations as a persistent and systemic phenomenon while using state-of-the-art LLMs for code generation, and a significant challenge which deserves the research community's urgent attention.

1 Introduction

Recent advances in generative AI, powered by Large Language Models (LLMs) like GPT-4 [1] and LLaMA [60], have revolutionized AI capabilities across modalities, excelling in a wide range of tasks such as image synthesis, text generation,

and natural language understanding. One such application is code generation, which is typically accomplished by first training or fine-tuning an LLM using vast amounts of programming data found on online repositories (e.g., GitHub), technical forums, and documentation. Both commercial/black-box (e.g., GPT-4 [1], Claude [3]) and open-source (e.g., CodeLlama [53], DeepSeek Coder [14]) varieties of such code-generating LLMs are readily available and are extensively used by both novice and expert programmers in their coding workflows to increase productivity. Recent studies indicate that up to 97% of the developers are using generative AI to some degree and that approximately 30% of code written today is AI-generated, reflecting significant perceived gains in efficiency and convenience [36, 54].

One critical shortcoming of LLMs is a phenomenon referred to as *hallucination*. Hallucinations are outputs produced by LLMs that are factually incorrect, nonsensical, or completely unrelated to the input task. Hallucinations present a significant obstacle to the effective and safe deployment of LLMs in public-facing applications due to their potential to generate inaccurate or misleading information. As a result, there has been increased efforts to research the detection and mitigation of hallucinations in LLMs [17, 22]. However, most existing research has focused only on hallucinations in classical natural language generation and prediction tasks such as machine translation, summarization, and conversational AI [7, 19, 33, 45]. The occurrence and impact of hallucinations during code generation, particularly regarding the type of hallucinated content and its implications for code security, are still in the nascent stages of research. Recently, Liu et al. [39] have shown that popular LLMs (e.g., ChatGPT, CodeRL, and CodeGen) significantly hallucinate during code generation and have established a taxonomy of hallucinations in LLM-generated code.

In this work, we focus on a specific type of hallucination during code generation called *package hallucination*. **Package hallucination occurs when an LLM generates code that recommends or contains a reference to a package that does not actually exist.** An adversary can exploit package

hallucinations, especially if they are repeated, by publishing a package to an open-source repository with the same name as the hallucinated or fictitious package and containing some malicious code/functionality. As other unsuspecting and trusting LLM users are subsequently recommended the same fictitious package in their generated code, they end up downloading the adversary-created malicious package, resulting in a successful compromise. This compromise can then spread through an entire codebase or software dependency chain, infecting any code that relies on the malicious package. This is a variation of the classical *package confusion attack* that has been enabled by code-generating LLMs.

Package confusion attacks, through techniques such as typosquatting (i.e., creating packages with names similar to popular ones to deceive users) and name similarity, have been a long-standing issue in the open-source software community [27, 47, 55]. Package hallucinations by code-generating LLMs threaten to exacerbate the problem by exposing an additional threat surface for such attacks. Trivial cross-referencing methods (i.e., comparing a generated package name with a list of known packages) are ineffective for detecting a package hallucination attack, as an adversary may already have published the hallucinated package with malicious code. Open-source repositories make no guarantee about the safety of hosted content; the mere presence of a package in an open-source repository does not confirm its credibility. A recent blog post [29] suggests that LLMs are prone to package hallucinations and provides a first approximation of their prevalence, but the extent to which this phenomenon occurs in state-of-the-art (SOTA) commercial and open-source LLMs, the nature of these hallucinations, and the effectiveness of potential mitigation measures have not been thoroughly investigated before.

In this paper, we conduct the first systematic study of the frequency and nature of package hallucinations across a variety of code-generating LLMs, operating under a diverse set of model settings and parameters. We specifically make the following novel contributions:

- **Characterizing the prevalence of package hallucinations by code-generating LLMs and related functional attributes:** We first comprehensively analyze the prevalence of package hallucinations in Python and JavaScript code generated by popular commercial and open-source LLMs. We also examine and characterize commonly observed LLM behaviors related to package hallucinations, including hallucination repetition, output verbosity, and the ability of these models to detect their own hallucinations.
- **Analyzing the effect of fine-grained changes to model settings on package hallucinations:** We further study how specific model settings, such as training data recency, model temperature, and decoding strategies affect the occurrence and nature of package hallucinations.
- **Characterizing common traits of the generated hallucinated packages:** We carefully study several key properties of the hallucinated packages, such as their semantic similarity to popular packages, their propensity of occurring across different models, and the influence of packages that were recently removed/deleted (from the corresponding repositories) on the hallucination rate, among others.
- **Testing of mitigation strategies:** We propose and comprehensively evaluate several techniques to effectively mitigate package hallucinations in LLM-generated code while maintaining the ability to produce effective code.
- **Publicly-accessible datasets for advancing research:** We make publicly available two novel datasets (one Python and one JavaScript) of 19,500 coding prompts for a wide range of coding tasks and 586,000 generated coding samples for fine-tuning/analysis.¹

2 Background and Related Work

In this section, we provide a brief background on open-source software security, code-generating LLMs, and the issue of hallucinations in LLMs.

2.1 Background

Package Confusion Attacks in Open-source Software Repositories. Modern software development has seen an increased reliance on open-source software packages and libraries that are publicly-available on centralized repositories. Many modern programming languages now rely on such centralized package repositories, with PyPI [9] (for Python) and npm [48] (for JavaScript) being the two most popular repositories. The open nature of these repositories, where anyone can upload new code packages/libraries, makes them an attractive platform for malware distribution. For instance, a total of 245,000 malicious code packages were discovered in open-source software repositories in 2023 alone [54].

Once a malicious package is uploaded, adversaries employ various techniques to trick users into downloading it, thereby integrating it into their codebases and dependency chains. These attacks often involve deliberately naming malicious packages to mimic legitimate ones, a tactic known as a *package confusion attack* [47]. Package confusion attacks can be broadly categorized into *typosquatting*, *combosquatting*, *brandjacking*, and *similarity* attacks [28], and are distinct from other types of software supply chain attacks such as corrupting legitimate packages or developing unique malicious packages from scratch as part of a long-term campaign. More than 1,200 package confusion attacks have been documented in the last six years [47], including the notable PyTorch compromise [55] and the Lazarus Group campaign [27].

¹All code and datasets can be found at: <https://zenodo.org/records/14676377> or <https://github.com/Spracks/PackageHallucination>

Packages/libraries often rely on other packages to function, thus creating extensive *dependency trees*. Infecting a single package in this dependency chain can be sufficient to compromise an entire software product or ecosystem [26, 47]. Public OSS repositories such as PyPI and npm have implemented various measures, including two-factor authentication, namespace protection, and software signing to mitigate the distribution of malicious packages [61, 66]. However, it remains unclear whether these repositories utilize any scan-based techniques for detecting malicious code, and they often do not disclose the full list of removed packages.

Automated Code Generation using LLMs. Modern LLMs continue to demonstrate advanced source-code generation capabilities, with success rates in correctly answering coding prompts surging from 25% in June 2021 to 96% by April 2024 [6]. With the increasing use of these models for software development, concerns are increasing about the likelihood of producing insecure or incorrect code that could create vulnerabilities in deployed applications. Early versions of code-generating LLMs were found to generate code containing vulnerabilities listed in the MITRE Top-25 Common Weakness Enumeration (CWE) 40% of the time [51]. Moreover, recent research has shown that AI-assisted programming not only results in less secure code, but also instills a false sense of security among developers [52].

Hallucinations by LLMs. It has been well documented that LLMs can unintentionally produce harmful information [41, 64], be manipulated for malicious purposes [15, 25], expose private information [34], and carry inherent biases in their training data [11]. A related phenomenon is hallucinations, where LLMs generate misleading or entirely fictitious information. These errors take various forms: the model might misinterpret the intended input (*input-conflicting hallucination*), produce inconsistencies with previous output (*context-conflicting hallucination*), or contradict established facts (*fact-conflicting hallucination*) [67]. Hallucinations can arise from three main root causes: (i) *data*, (ii) *training*, and (iii) *inference* [18]. Data-related hallucinations occur when the source data itself is flawed with misinformation [37], bias [11], or incomplete records [50]. Architecture flaws [38] or suboptimal training objectives [62] during training could also result in downstream hallucinations, while inference time issues such as defective coding strategies [16] and imperfect decoding representations [5, 42] are other contributors. The probabilistic nature of LLMs presents a challenge in mitigating hallucinations. This nondeterminism, while it fosters creativity and generates diverse and innovative content, also contributes to the generation of hallucinated content. Balancing creativity with accuracy remains a central challenge in deploying LLMs, underscoring the complexity of developing effective mitigation strategies.

Package Hallucinations and Security Risks. Package hallucinations, a special form of fact-conflicting hallucinations,

are instances where LLMs generate fictitious (non-existent) or erroneous package names in the generated source code. As outlined earlier, an adversary can quickly create malicious packages (on the appropriate open-source repository) with the same name as these hallucinated packages, thus effecting a very simple, yet effective, form of package confusion attack. Unsuspecting users, who trust the LLM output, may not scrutinize the validity of these hallucinated packages in the generated code and could inadvertently include these malicious packages in their codebase [52]. This resulting insecure open-source code also has the potential of being included in the dependency chain of other packages and code, leading to a cascading effect where vulnerabilities are propagated across numerous codebases. The simplicity and scale of such LLM-enabled package confusion attacks highlight the critical need for quantifying this existing risk, understanding the nature of this unique type of hallucination, and developing effective mitigation techniques that maintain the utility of the code generated by the LLMs. This is precisely what we aim to accomplish in this work.

2.2 Related Work

The possibility of code-generating models recommending malicious or typosquatted packages was first suggested in 2021 as tools such as GPT-3 and Codex were released as viable code generation platforms [4]. At the time, the risk of these tools suggesting vulnerable, malicious, or typosquatted packages was assessed to be low [6]. In particular, the related but distinct concept of package hallucinations was not explicitly considered in this initial risk assessment; either because such an attack scenario was not considered at all or because the threat was thought to be negligible. The capabilities of generative AI agents have advanced significantly since that introductory evaluation.

Although a comprehensive study on the prevalence of package hallucinations in LLM-generated code has not been done previously, a recent blog post by Lanyado [30] outlines the results of some preliminary tests on commercial LLMs such as GPT, Gemini, and Cohere. Their testing confirms the presence of hallucinated packages in the code generated by these LLMs, but their initial estimate of hallucination rate was $5\times$ higher than our findings, and they do not consider popular open-source LLMs or study possible mitigation approaches. They also fail to systematically characterize the generated hallucinated packages and model properties that have an impact on hallucinations.

In contrast, we conduct a rigorous and comprehensive evaluation across a broader range of models, including the first analysis of package hallucinations in open-source models of any kind, at a scale that has not been previously done. To this end, we provide thorough testing with a larger custom dataset covering two programming languages (namely, Python and Javascript), followed by a detailed analysis of the significant

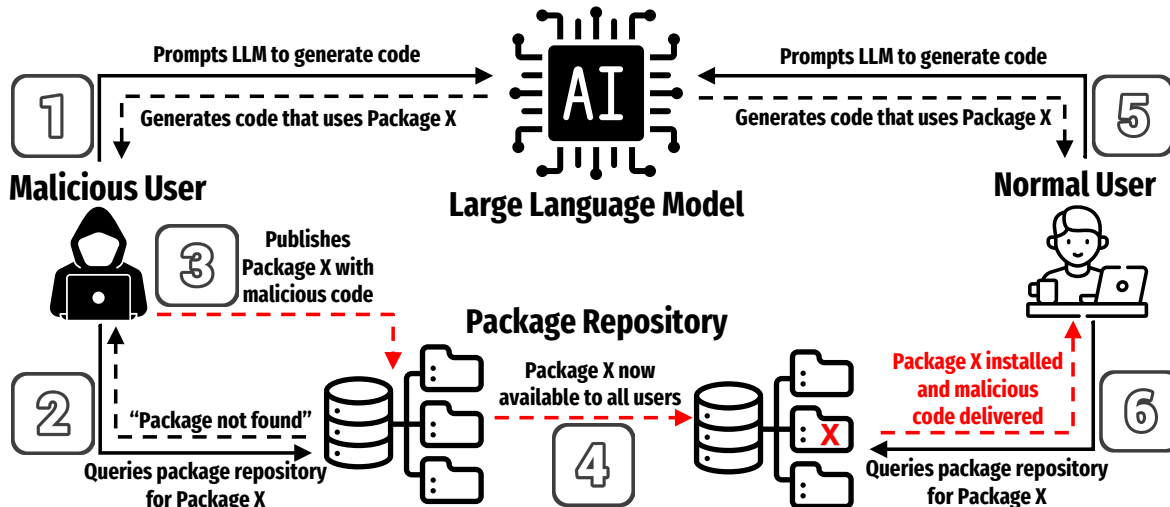


Figure 1: Exploiting Package Hallucination.

characteristics of this phenomenon.

3 Research Questions

Adversary Model and Assumptions. We assume an adversary who wants to execute a package confusion attack by leveraging package hallucinations in the code generated by closed-source and open-source code-generating LLMs (see Figure 1). The target of the adversary are users who employ such LLMs for generating code. Here we assume that the LLM generates code that requires additional packages for the user to install, which the user does without sufficient verification of the recommended packages. In other words, the target users fully trust the LLMs to include only valid package names in the generated code. We assume that the adversary has access to the same set of LLMs for code generation as the target users, and is unable to modify or manipulate the model and model parameters of these LLMs (e.g., via retraining or fine-tuning) before they are used by the victims. The adversary is able to determine a list of hallucinated packages generated by these LLMs (for example, by cross-referencing the package repository), and then is able to realize a package confusion attack by creating a package of the same name on the corresponding package repositories. These newly created (and now publicly available) packages by the adversary could contain malicious code or functionality. Research has shown that installing open source Python or JavaScript packages allows the execution of arbitrary code by an attacker [49]. Previous work [30] has also established the viability of such an attack by publishing a hallucinated package to an open source repository and demonstrating that the package is actively downloaded and was incorporated into the dependency chains of other packages/code.

Research Questions. We now organize our investigation into

the following five broad Research Questions (RQ).

RQ1: How prevalent are package hallucinations while generating Python and JavaScript code using LLMs? Our aim here is to thoroughly examine how often package hallucinations occur with both widely-used commercial and open-source LLMs when they generate Python and JavaScript code across various programming tasks.

RQ2: How are package hallucinations impacted by select model settings? Here our goal is to comprehensively analyze how training data and decoding strategies impact the package hallucinations produced by these code generation LLMs.

RQ3: What are the commonly observed model behaviors related to package hallucinations? This RQ will exhaustively study model behaviors such as hallucination repetition by a single LLM (hallucination persistence) and across multiple LLMs (cross-model hallucinations), output verbosity, and the ability of LLMs to detect their own hallucinations (after generation).

RQ4: What are some of the defining properties/attributes of the observed package hallucinations? The goal of this RQ is to analyze the properties of the hallucinated packages such as semantic similarity between hallucinated and popular packages, number of cross-language hallucinations (i.e. non-existent packages from the language requested but valid packages in another programming language), and the number of generated packages that were recently removed from the source repositories.

RQ5: Is it possible to effectively mitigate package hallucinations using best practices in the literature and knowledge gained from earlier results? Through this RQ, we will investigate if code-generating LLMs can be designed to reduce hallucinations with minimal compromise to code quality. In this direction, we will study if techniques such as retrieval

augmented generation (RAG) [32], self-detected feedback, decoding strategies, and supervised fine-tuning [57] are effective package hallucination reduction strategies.

4 Experiment Design

To address the RQs outlined above, we design several experiments to repeatedly prompt LLMs to generate code and then analyze the generated code. Our experimentation pipeline consists of three distinct phases: (i) *prompt dataset generation*, (ii) *code generation*, and (iii) *hallucination detection*, each of which is described next.

4.1 Prompt Dataset

The experiments are designed to exhaustively test each LLM through a complete range of coding tasks. Existing benchmark datasets of coding prompts contain only a limited number of prompts (e.g. only 164 prompts for both EvalPlus [40] and HumanEval [6]) and lack diversity. Therefore, we create a new code prompt dataset for our experiment that contains both breadth and depth in terms of overall number of prompts and range of topics. Our goal was to develop a dataset that accurately and comprehensively represents the coding tasks commonly requested by everyday users. To accomplish this, we employ two distinct approaches, as described below.

Stack Overflow Dataset. To model the input prompts around real programmer questions, our first prompt dataset was created using Stack Overflow [56] questions across relevant programming topics and subject areas. Stack Overflow is a popular online question-and-answer service for software programmers and developers. To capture a wide range of topics, we utilize the “tag” feature of Stack Overflow, which allows users to label posts according to a subject matter. We included any tag that had more than 5,000 questions and was also relevant to Python or JavaScript (the two programming languages that we focus on in this work, as detailed in Section 4.2). For each of the 240 manually selected tags that met this criterion (a full list of tags can be found with the paper artifacts 1), we extracted the 20 most upvoted questions, resulting in 4,800 prompts (i.e., 4,800 prompts for Python and 4,800 prompts for JavaScript).

As more recent data is less likely to be included in the pre-training data of LLMs, we are also interested in investigating the temporal correlation between data recency (i.e., how recently the question was asked on Stack Overflow) and model hallucination rate. To enable such an analysis, we ran two queries on Stack Overflow; one that captured only the most popular questions in the selected tags from 2023 and another that captured the most popular questions for all years prior to 2023. By including the two different ranges of time, we effectively doubled the original number of prompts, for a total of 9,600 for each of the two languages.

Not all questions asked on Stack Overflow may involve coding or require code to answer the question. Rather than attempting to filter out such prompts during the code generation phase, which is non-trivial and error-prone, the LLM is asked to answer the question and only provide code if necessary. In the end, this may result in a slightly lesser number of usable LLM-generated code samples but is more realistic as LLMs are expected to accommodate imperfect user inputs.

LLM-generated Dataset. As a majority of the programming tasks require some library/package, our next idea was to use the package repositories themselves as a good representation of the full spectrum of coding topics. Our goal was to represent as many code generation tasks as possible in one comprehensive dataset. We take the 5,000 most popular Python and JavaScript packages (based on the number of downloads) and scrape the official package description as listed on PyPI and npm, respectively. These descriptions are then individually inputted to the Llama-2 70B model with instructions to generate a coding prompt based on the package description (the exact prompt available in Figure 13). This process generated roughly 4,800 prompts for Python and JavaScript each, resulting in two datasets of approximately the same size (some packages with no description or descriptions in a non-English language were discarded). Similarly to the Stack Overflow dataset, we doubled the LLM-generated dataset for temporal analysis by dividing it into two segments: the packages most downloaded in 2023 and the packages most downloaded prior to 2023. When a package appears in both sets, we remove the package from the latter set to ensure that there is no overlap. Removing duplicates from the latter dataset guarantees that the remaining packages will be those that have increased in popularity during the last year, capturing the desired signal. A truncated list of the LLM-generated dataset can be found in Appendix A.

For brevity, the two temporally distinct datasets, one from the past year and one from before 2023, will be referred to as the ‘recent’ and ‘all-time’ datasets for the remainder of this paper.

4.2 Code Generation

Model Selection. For our experiments, we chose the models that were the highest ranked on the EvalPlus leaderboard (as of January 20, 2024) [40]. During the creation of our model list, we ignored the fine-tuned versions that were ranked below their corresponding foundational models and only selected one fine-tuned version of the same foundational model of the same parameter size [40]. EvalPlus maintains a ranking of the top performing LLMs for code correctness according to a rigorous code synthesis evaluation framework. Our goal was to include a mix of top-performing base models and a few of the best-performing fine-tuned variants. We also included the GPT series of models (GPT-3.5, GPT-4, and GPT-4 Turbo) in

our experiments, which currently hold the top rankings on the leaderboard. GPT models are widely considered as SOTA in terms of code generation models at the time of writing and add value to our experiments as representative commercial models. The models were not modified or altered in any way prior to testing; they are strictly “off-the-shelf.” Table 1 provides a complete list of the models that we tested in our experiments.

Table 1: Details of the models that were evaluated.

Model	Parameters	License	Open Source
ChatGPT 4.0 [1]	Unknown	Commercial	✗
ChatGPT 4.0 Turbo [1]	Unknown	Commercial	✗
ChatGPT 3.5 Turbo [4]	Unknown	Commercial	✗
CodeLlama [53]	7B, 13B, 34B	Free	✗
DeepSeek [14]	1.3B, 6.7B, 33B	Free	✓
MagiCoder [65]	6.7B	Free	✓
WizardCoder [43]	34B	Free	✓
Mistral [23]	7B	Free	✓
Mixtral [24]	8x7B	Free	✓
OpenChat [63]	7B	Free	✓
WizardCoder-Python [43]	7B	Free	✓
CodeLlama-Python [53]	33B	Free	✓

Language Selection. In our experiments, we focus on two of the most popular programming languages, JavaScript and Python. These languages were chosen due to their overall popularity (#1 and #2 according to the GitHub 2023 Octoverse report [12]) and their dependence on open-source repositories for package management. Other popular programming languages like Java, C, or C++ do not rely on a centralized open-source repository, as Python and JavaScript do, which is a key component of this vulnerability. The open-source package repositories for these languages, npm and PyPI, represent ecosystems of 5.1 million and 573 thousand packages, respectively [58]. Of the 16 total models tested (see Table 1), 14 were tested for both Python and JavaScript, while two fine-tuned Python-specific models, WizardCoder-Python and CodeLlama-Python, were only tested for Python.

Testing Environment. All open-source models were tested using the Hugging Face `transformers` package and quantized versions of the models, which reduces parameter precision to boost inference speed and lower memory use without significantly impacting performance. Specifically, the GPTQ quantization method was used, which utilizes a one-shot weight quantization method based on approximate second-order information that has a negligible effect on the accuracy of models, making it an ideal choice [10]. Additionally, quantized models better simulate the performance that a typical user can expect when running models on commercial grade hardware, making them more accessible and practical for everyday use.

For testing uniformity, we use the same parameters and

quantization precision for all open-source models, which are summarized in Appendix C, along with the computing environment used. To generate code for our analysis, we query each LLM (Table 1) with prompts from the two datasets along with a system message which contains specific instructions regarding the task and output format. An overview of the process, including the system messages used during each step, is detailed in Appendix B. The experiment generates 19,200 code samples per model (16 Python tests + 14 JavaScript tests * 19,200 = 576,000 total code samples), which are further analyzed to determine which packages are required to execute the generated code.

4.3 Detection Methodology and Heuristics

To detect hallucinated packages, we first need to extract package names from the LLM output or the generated code sample, which is non-trivial. Simply parsing the code for “import” or “require” is not useful, as the arguments in those statements refer to modules and not packages. There is no way to definitively determine the required packages from a code snippet alone. A detailed explanation of this problem can be found in Appendix G. To solve this problem, we employ the following three heuristics to determine/identify package names in the generated code:

Heuristic 1. As part of our first heuristic, we parse the generated Python and JavaScript code for “`pip install`” and “`npm install`” commands, respectively. These commands look for the specified package in the PyPI/npm repository, resolve its dependencies, and install everything in the current Python/JavaScript environment to ensure that future module requests will work. This is the most straightforward heuristic for detecting package names (and thus hallucinations), as it involves explicit commands from the code generation model for package download/installation. This is significant because if the referenced hallucinated package was indeed used by an adversary to execute a package confusion attack, it could immediately trigger download/install of the malicious code in the package. Note that we did not directly ask the model to provide these commands, but allowed them to occur naturally during the generation process. As such, we observed that these instances (“`pip install`” and “`npm install`”) occur for 7% of the total output.

Heuristic 2. For the second heuristic, each generated code sample is used as input to the same model that generated it. The model is then prompted for a list of packages that would be required to run the given code. Our intuition is to mimic an actual user/developer who is using LLMs for code generation. If the user gets an error due to an uninstalled package when attempting to execute the generated code, they could query the model for the correct package to install. We wanted to replicate this intuitive process to identify the package names required by the generated code.

Heuristic 3. As the third heuristic, we reuse the original prompt used to generate the code sample as an input to the model and ask the model to output package names that would be required to accomplish this coding task. Similarly to the previous heuristic, this process of extracting package names simulates another approach users would take to obtain package names from the model that generated the code, if the required packages were not mentioned in explicit “pip install” and “npm install” commands.

Once each model provides specific package names (through the three heuristics outlined above), we simply compare each package name to a master list of package names acquired from PyPI and npm, respectively (each list is as of 10 January, 2024). If a package name is not on the master list, it is considered a hallucination. We acknowledge the possibility that the master list of packages obtained from the package repositories has already been contaminated with malicious hallucinated packages. It is not possible to guarantee that the master list actually represents the ground truth of valid packages; however, the presence of hallucinated packages already in the master list would actually produce fewer hallucinations, and therefore our results represent a lower bound of hallucination rate.

5 Evaluation Results

In this section, we present the results of our experimental analysis related to RQ1 – RQ4. After using both Python and JavaScript for RQ1, for RQs 2 through 4, we focus our analysis only on the Python programming language, a subset of the original models tested, and randomly sampled subsets of our original datasets. Given the consistent results that we were able to obtain across both languages for RQ1, we believe that this narrowed scope of discussion for RQs 2 through 4 should not compromise the generalizability of the conclusions and would allow for a deeper analysis of package hallucinations in a controlled setting. We selected GPT-4 Turbo, GPT-3.5, CodeLlama 7B, and DeepSeek 6.7B for the in-depth analysis of RQs 2-4, representing the best-performing and most popular open-source models.

5.1 Prevalence of Package Hallucinations (RQ1)

In our first experiment, our goal was to quantify the prevalence of package hallucinations across different models by generating and analyzing a large number of code samples. We conducted 30 tests (using 16 models for Python and 14 models for JavaScript, as described in Table 1) producing a combined 576,000 code samples using both the Stack Overflow and LLM-generated datasets (Section 4.1). Each code sample was evaluated for hallucinations according to the heuristics defined in Section 4.3, which include parsing the generated code and prompting the model for packages twice per code sample,

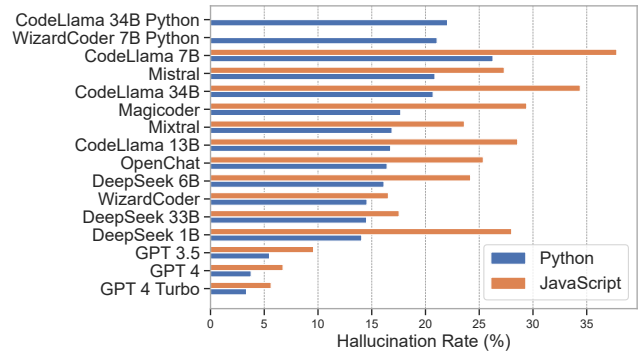


Figure 2: Observed hallucination rates of the tested models.

for a total of 1,152,000 package prompts across all tests. To measure LLMs’ propensity to produce hallucinated packages during code generation, we use the *package hallucination rate* metric, which can be expressed as a simple ratio of the number of hallucinated packages to the total number of recommended packages. The total hallucination rates for each evaluated model are presented in Figure 2. More fine-grained results on hallucination rates for all models tested, covering both Python and JavaScript, are presented in Appendix E.

These 30 tests generated a total of 2.23 million packages in response to our prompts, of which **440,445 (19.7%) were determined to be hallucinations, including 205,474 unique non-existent packages** (i.e. packages that do not exist in PyPI or npm repositories and were distinct entries in the hallucination count, irrespective of their multiple occurrences). Our results for GPT-3.5 (5.76%) and GPT-4 (4.05%) differ significantly from previous work on package hallucinations [30], which found hallucination rates 4–6 times higher (24.2% and 22.2%, respectively) for those specific models. GPT series models were found to be 4 times less likely to generate hallucinated packages compared to open-source models, with a hallucination rate of 5.2% compared to 21.7%. GPT-4 Turbo resulted in the lowest overall hallucination rate at 3.59%, while DeepSeek 1B had the best hallucination rate among open-source models at 13.63%. Python code resulted in fewer hallucinations than JavaScript (15.8% on average compared to 21.3% for JavaScript). Despite the difference in hallucination rate between the two languages, there is a linear relationship between the results (as shown in Figure 14 in the appendix), demonstrating that the propensity of a model to hallucinate is positively correlated between programming languages. The above results provide strong evidence that package hallucinations are a pervasive issue across all code-generating LLMs.

RQ1 Summary: Package hallucinations were found to be pervasive phenomenon across all 16 models tested. Commercial models hallucinated 4× less compared to open-source models. Python code resulted in a lower hallucination rate compared to JavaScript.

5.2 Impact of Model Settings (RQ2)

Effect of Temperature Settings. The temperature setting in a LLM is used to adjust the randomness of the generated responses, where a lower temperature results in more predictable and deterministic outputs, while a higher temperature increases creativity and diversity in the responses (Appendix F.1). We varied this setting for each model between the minimum and maximum allowed values and observed the change in hallucination rate (the maximum temperature for the GPT series models is limited to 2, while the open-source models can be set to 5). All models exhibited a **clear increase in hallucination rate as temperature value increases**, with the effect becoming severe at maximum values. The OpenAI models, as shown in Figure 3, showed only a slight increase in hallucination rate between temperatures 0 and 1, which then increased sharply between 1 and 2. In particular, GPT-4 resulted in a hallucination rate (8.9%) nearly 4 times lower than GPT 3.5 (31.8%) at its maximum temperature. At the highest temperature values, open-source models start to generate more hallucinated packages than valid packages. Most LLMs operate at a default temperature in the range of 0.7 to 1, however, our results indicate that a lower temperature value can reduce package hallucinations, with the optimal value varying per model. Lower temperature also yields more deterministic responses, presenting a trade-off between risk of hallucination and creativity. Therefore, selecting the appropriate temperature value is not a straightforward decision.

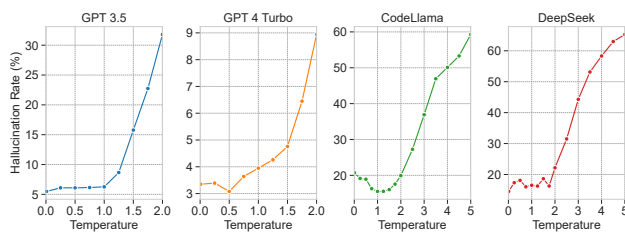


Figure 3: Hallucination rate vs. temperature.

Effect of Decoding Strategies. Next, we adjusted several decoding parameters (top- p , top- k , and min- p values) to reduce the chances of a low probability token being selected as a potential package, with the intuition that lower probability tokens correspond to higher probabilities of hallucination in this context. The following is a summary of the parameters and values we modified.

- Top- p (0.4, **0.6**, 0.8): Tokens with probabilities adding up to less than this number are discarded.
- Top- k (5, **10**, 15): Select only the top- k most likely tokens.

- Min- p (0.1, **0.2**, 0.3): Tokens with probability smaller than (min- p * probability of most likely token) are discarded.

We evaluated each listed value in isolation, followed by a combined evaluation of the values highlighted in bold, resulting in a total of 10 tests. Note that top- k and min- p were only tested for DeepSeek and CodeLlama, as these values are not modifiable through the OpenAI API. Varying the decoding values induced a slight *increase* (1.16% on average) in the hallucination rate for the four models across all values tested. As we will expand on in RQ3, package hallucinations are often persistently repeated across many iterations.

This suggests that greedy decoding strategies, which prioritize the most probable tokens (i.e., the most probable token is always selected), would still generate fictitious packages. This differs from other types of hallucinations, which generally occur when low-probability tokens are sampled. This persistent nature of package hallucinations highlights the inherent complexity of the problem.

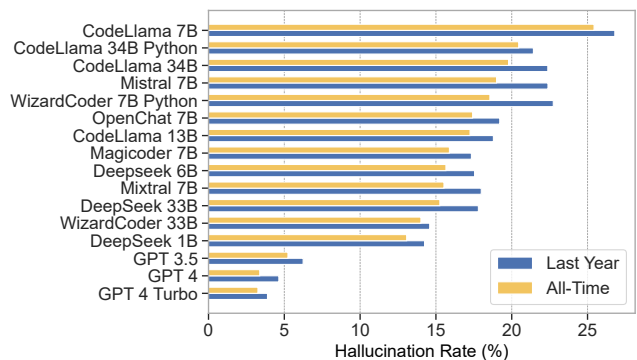


Figure 4: Hallucination rates of recent vs. all-time data sets.

Recency of Subject Matter. As described in Section 4.1, we separated our coding prompts into two temporal datasets to evaluate whether the hallucination rate was correlated with topics/packages that emerged after the model was trained. A lower difference between the rates of recent and all-time prompts indicates better performance in handling questions that fall outside the model's pre-training data and therefore a more generalizable model. The models we tested were shown to be more likely to generate a package hallucination when responding to prompts that deal with more recent topics. This difference resulted in a 10% higher hallucination rate on average for older data versus more recent data.

Overall, all 16 Python models we evaluated **demonstrated a higher hallucination rate when being prompted about questions or packages that were popular within the past year** (Figure 4). These higher rates are at least partially due to the inherent limitations and training costs of modern LLMs. As noted in the OpenAI GPT-4 technical report [1], LLMs cannot update themselves with new information after release

and have no knowledge of the world beyond their training data cutoff date. Although fine-tuning can enhance specific tasks, it generally does not improve the model's overall knowledge of the world. The massive cost of training LLMs from scratch, evidenced by the 1,400,000 GPU hours (220 years) required to train the 12 CodeLlama models, makes continuously updating pre-training data impractically expensive [53]. This cost, along with steadily increasing model sizes and training times, poses a significant barrier to reducing package hallucinations for advanced coding prompts and packages.

RQ2 Summary: Lower temperatures result in the lowest hallucination rate while hallucination rates increase dramatically with temperature values larger than 1. Altering decoding and sampling parameters in the model does not improve hallucination rates. More recent prompting topics lead to a 10% increase in hallucination rates.

5.3 Model Behaviors (RQ3)

Frequency of Repeated Hallucinations. To determine whether hallucinations are random error or repeatable phenomena, this test focuses on the persistence of hallucinations within a model. We randomly sampled 500 prompts that generated package hallucinations during our initial testing and then repeated those queries 10 times per prompt. Of those 10 queries, we recorded how many times the original hallucinated package was regenerated. Our analysis reveals an unexpected dichotomy when repeatedly querying a model with the same prompt that generated a hallucination: 43% of hallucinated packages were repeated in all 10 queries, while 39% did not repeat at all across the 10 queries. This is indicated in Figure 5, which shows prominent spikes at zero repetitions and at 10 repetitions, respectively, for all models. In addition, 58% of the time, a hallucinated package is repeated more than once in 10 iterations, which shows that a majority of hallucinations are not simply random errors, but a repeatable phenomenon that persists across multiple iterations. This is significant because a persistent hallucination is more valuable for malicious actors looking to exploit this vulnerability and makes the hallucination attack vector a more viable threat.

Verbose Models versus Conservative Models. LLMs operate with inherent randomness and uncertainty. This behavior enables novel and creative output, a desired feature for many NLP tasks but less welcome for code generation, which requires a high degree of accuracy and must adhere to rigid syntax. We define a verbose model as one that operates with higher degree of uncertainty and randomness by generating a greater number of distinct package names while a conservative model generates a lesser number of distinct packages, generally using only the most popular and well-known packages. To this end, we investigated whether verbose models correspond to a higher rate of package hallucinations. Our results (Figure 6) show a **correlation between hallucination rate**

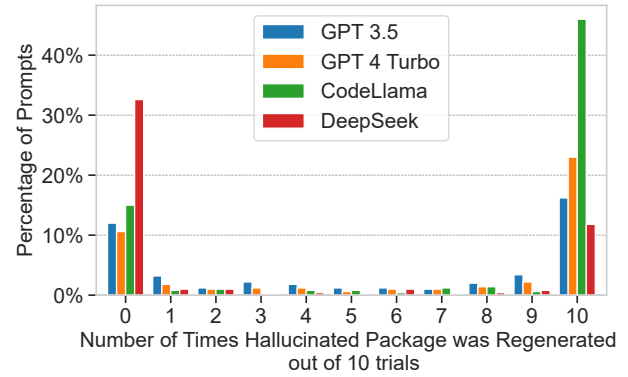


Figure 5: Frequency of an identical hallucinated package name generated from the same prompt across 10 trials.

and number of unique packages that were recommended during this experiment (i.e. a more verbose model was associated with a higher hallucination rate). In light of these findings, it is reasonable to suggest that models generating code should adopt a more conservative approach (i.e. limiting package suggestions to a smaller list of well-known packages rather than generating names with uncertainty). The mod-

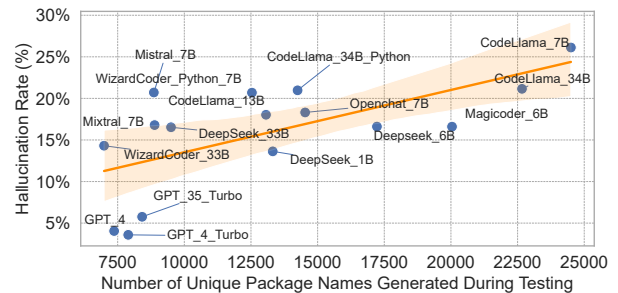


Figure 6: Unique packages vs. total hallucination rate.

els with the lowest hallucination rates in our study adhered to a smaller subset of packages when generating code, and these models (e.g., the GPT series) also scored the highest on the EvalPlus [40] code quality benchmarks. This suggests that improving code quality and reducing hallucinations can potentially be achieved simultaneously without a trade-off.

LLMs' Ability to Detect Hallucinations. We then evaluated each model's ability to identify hallucinations versus valid packages, both from its own code generation outputs and those generated by other models. To test this, we conducted two binary classification tests: (i) each model's ability to detect hallucinated packages from its own generated code and (ii) each model's ability to detect hallucinated packages from code generated by other models. The names of the valid and hallucinated packages produced by each model were randomly sampled, and each model was asked "Is [package name] a valid Python package?" Identification accuracy was

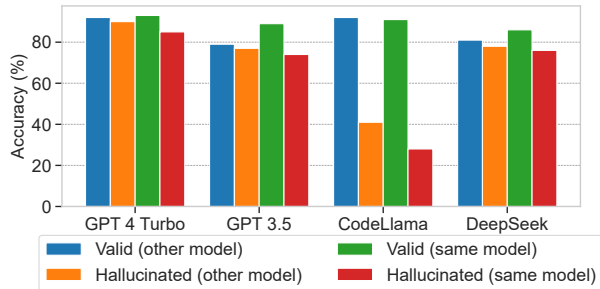


Figure 7: The ability of models to correctly identify valid vs. hallucinated packages.

calculated as the ratio of correct identifications to the total number of packages provided.

Figure 7 shows that 3 of the 4 models (GPT 4 Turbo, GPT 3.5, and DeepSeek) proved to be **highly adept in detecting their own hallucinations** with detection accuracy above 75%. Table 2 displays the recall and precision values for this test, with similarly strong performance across the 3 proficient models. This phenomenon implies that each model’s specific error patterns are detectable by the same mechanisms that generate them, suggesting an inherent self-regulatory capability. The indication that these models have an implicit understanding of their own generative patterns that could be leveraged for self-improvement is an important finding for developing mitigation strategies. CodeLlama displays unique and inter-

Table 2: Performance of hallucination detection tests.

Model Name	Other		Same	
	Precision	Recall	Precision	Recall
GPT 4 Turbo	0.91	0.91	0.89	0.89
GPT 3.5	0.78	0.78	0.82	0.82
CodeLlama	0.72	0.66	0.66	0.60
DeepSeek	0.80	0.80	0.81	0.78

esting behavior during both tests, as it has an overwhelming propensity to label most packages as valid, resulting in a lower accuracy for hallucinated packages.

RQ3 Summary: Package hallucinations are often persistently generated. Models that generate fewer packages when prompted are correlated with a reduced hallucination rate. Several models were able to detect their own hallucinations with greater than 75% accuracy.

5.4 Characteristics of Hallucinations (RQ4)

Occurrence of the Same Package Hallucination Across Different Models. To analyze the possibility that the same hallucinated packages are generated across different models, we measured how many models generated the same pack-

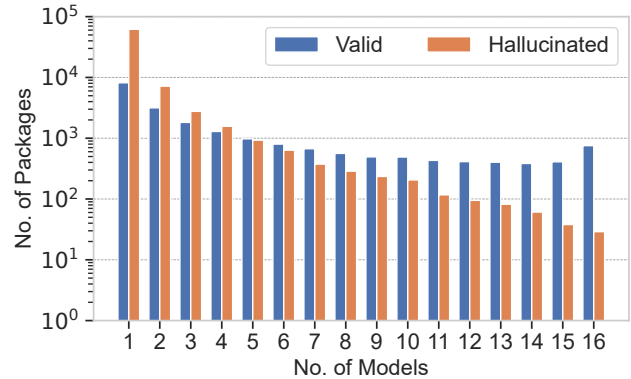


Figure 8: Number of models in which each unique package (valid & hallucinated) appeared, with the y-axis on a log scale.

age name given a confirmed package hallucination. Figure 8 shows a clear pattern in which **a large majority (81%) of distinctly generated package names were generated by only one model**. In other words, the specific package names were usually unique to a single model, where only the most common packages were generated by more than one model. The two populations (valid and hallucinated packages) diverge as the number of models increases, with the number of hallucinated packages decreasing nearly exponentially and the distribution of valid packages becoming more uniform. The finding that valid packages are less dependent on the specific model used for generation is attributable to their frequent appearance in training data and applicability to universal coding problems, leading to their widespread use in a broad range of prompts.

Combining the insights gained during the persistence analysis (Figure 5) leads to a key observation. As previously shown, hallucinations are often persistent (58% are repeated within 10 iterations) within the same model but are not often repeated between models, as 81% of hallucinated packages are generated by only one model. This further reinforces the evidence that while hallucinations are a common phenomenon across various models, the exact nature of these hallucinations is generally model specific. This behavior is particularly surprising given that our testing includes multiple models from the same family (i.e. 3 GPT models, 4 CodeLlama models, and 3 DeepSeek models). These models presumably use the same training data for each version, yet each model generates unique hallucinations that are not found in other models.

Semantic Similarity Between Hallucinated and Popular Valid Packages. In order to analyze the semantic similarity between hallucinated and popular real/valid packages, we measured the average *Levenshtein distance* of a package to its nearest neighbor (i.e., the closest valid package). Levenshtein distance is a measure of how many insertions, deletions, and substitutions are required for two strings to match [31]. If the distribution of Levenshtein distances is skewed heavily

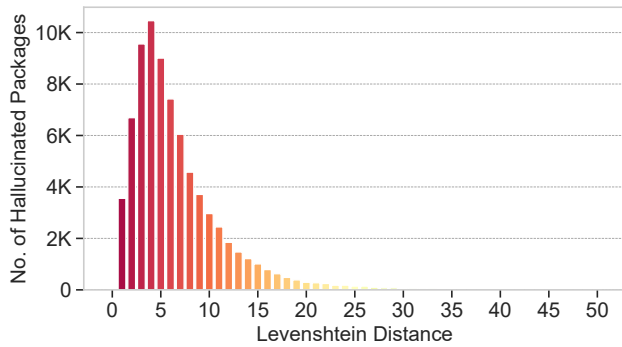


Figure 9: Levenshtein distance of hallucinated packages to nearest valid package.

right, with a peak at or near 0, this would indicate that most hallucinations are very similar to valid package names. In that case, attackers could infer a hallucination target based on more traditional package confusion methods (e.g., typosquatting) rather than analyzing a large volume of model output over time to detect persistent hallucinations that could be used as vessels for malicious code. A higher distance reflects that package hallucinations are more random in nature and difficult to predict, rather than the result of minor grammatical errors.

The results of the Levenshtein distance test, as seen in Figure 9, suggest that most package hallucinations are not simple *off-by-one errors*. An off-by-one error in our case refers to a discrepancy between the hallucinated package and its nearest match, involving a difference of 1 to 2 characters, including numbers, letters, or punctuation marks. Our results show that only 13.4% (10,263 of 76,489) have a Levenshtein distance of 1 or 2. Another 37.9% (29,025 of 76,489) of packages registered a score between 3 and 5, which would indicate two words with a common root word or concept that still differ significantly. In particular, 48.6% (37,207 of 76,489) of hallucinations scored 6 or higher, with 20.2% (15,457 of 76,489) of those scoring 10 or higher, indicating two strings that are very different and likely do not share any common theme.

The presence of such a large proportion of high Levenshtein values suggests that **the majority of hallucinations are not merely trivial typographical errors but are substantively different from existing package names**. The observed results provide further evidence that the root cause of hallucinations is likely to be more complex than minor string manipulation, pointing to deeper issues in the model’s generative processes that govern the creation of package names. The long right tail of the distribution in Figure 9 indicates a wide variety of hallucinations spread over a broad range, revealing a diversity in types of errors and reinforcing that the generation of hallucinations is a complex issue not limited to simple character substitutions, additions, and deletions.

Effect of Deleted Packages. To determine whether packages

Table 3: Confusion by programming language repository.

Programming Language	No. of Cross-Language Hallucinations
JavaScript (npm)	6,705
R (CRAN)	293
Rust (Cargo)	181
Ruby (Rubygems)	123
PHP (Packagist)	47
Swift/Objective C (Cocoapods)	10
.NET (Nuget)	9
Go (Go)	0
Java (Maven)	0

that existed before a model’s pre-training data cut-off date (i.e., the final day of data included in the model’s training set) but were subsequently removed contribute significantly to package hallucinations, we conducted an analysis using package download counts obtained via Google BigQuery [13]. We searched PyPI download counts from 2022 and earlier to compile a list of packages that existed before 2023 but are no longer available on PyPI. This list was then compared against the master list (obtained from PyPI as of January 10, 2024) of hallucinated packages across all models. We detected 12,871 packages that were available between 2020 and 2022 and have since been removed from PyPI. Of these deleted packages, only 133 (0.17%) were generated during our analysis, indicating that **deleted packages are a negligible source of package hallucinations**. This finding contradicts our hypothesis, as we expected a sizable percentage of hallucinated packages due to the presence of deleted packages in the training data.

Effect of Language Confusion. Another behavior observed during the main experiment is the tendency to confuse programming languages while generating package output (i.e. the model is asked to provide Python packages, but instead provides JavaScript packages). To validate the potential existence of this behavior in code-generating LLMs, we obtained master lists of packages (using libraries.io [58]) from the nine most popular open-source repositories and compared our list of hallucinated package names generated during Python testing to the respective master lists of valid packages from other programming languages. Any intersection between the two lists indicates a *cross-language hallucination*. **Overall, only JavaScript is a significant source of cross-language hallucinations, as 8.7% (6,705/76,489) of hallucinated Python packages are valid JavaScript packages.** All other languages contributed negligible hallucinations, combining for only 0.8% (663 of 76,489) across eight other open-source repositories, including R, Rust, Ruby, PHP, Swift and .NET (see Table 3 for complete results).

RQ4 Summary: Most unique hallucinated package names were generated by a single model and thus only appeared in 1 out of 16 tests. Most hallucinated package names were not semantically similar to a valid Python package as measured by the Levenshtein distance. Deleted packages were a negligible source of package hallucinations, while JavaScript was the only significant source of cross-language hallucinations.

6 Mitigation

Motivated by the finding of systemic package hallucination in all the tested models, we next investigate techniques to mitigate the occurrence of such hallucinations (thus addressing RQ5).

6.1 Mitigation Strategies

A straightforward approach to address package hallucinations could be to cross-reference a master list of valid packages with the model’s output, thereby eliminating any incorrect package names. This type of filtering method is ineffective as a defense strategy, as an attacker could immediately publish a hallucinated package to the repository and be subsequently included in the “allow” list. Although using a curated list of “known good packages”, using some metric such as package popularity, would be a more effective method, this is still considered a blunt and reactive approach that requires constant verification and updating. We refer to these types of filtering techniques as *post-generation techniques*, as they aim to filter hallucinated content after it has been generated by the model. In contrast, the techniques we introduce in this section are *pre-generation techniques* which aim to prevent the model from generating hallucinated content. These techniques proactively reduce the likelihood of hallucinations at the source with less overhead and maintenance while providing a more reliable output.

Developing specialized mitigation strategies for code-generating LLMs is an area that has not been extensively investigated; therefore, we draw upon general hallucination mitigation strategies designed for typical NLP tasks, which can also be adapted for code generation. These strategies can generally be grouped into two broad categories: *prompt engineering* and *model development* [59]. Prompt engineering includes methods such as *Retrieval Augmented Generation (RAG)*, *self-refinement*, and *prompt tuning*. RAG approaches involve enriching the original prompt with additional information gathered from an external source, such as the web or a pre-determined database [32]. This augmentation can occur at any stage — before, during or after response generation — and can be iterative, improving over multiple cycles until the response is verified to be accurate. Self-refinement strategies, on the other hand, utilize the model itself to detect and refine potential hallucinations.

The second main mitigation strategy involves improving the underlying LLM model itself through improved *decoding strategies* or *supervised fine-tuning*. Supervised fine-tuning alters model parameters to improve performance on tasks prone to hallucinations, using a labeled dataset for more precise training. Decoding strategies are also considered a viable mitigation strategy from the literature, but based on our findings from RQ2, we know that altering decoding parameters, such as *top-k*, *top-p*, and *min-p*, do not result in a decreased hallucination rate. We evaluate each of the three remaining categories to determine their applicability to the code generation task and to our specific use case of reducing package hallucinations during code generation.

6.2 Mitigation Implementation and Results

Retrieval Augmented Generation (RAG). We employ a method of RAG which supplements the prompt with valid package names before generation to assist the model in generating a response. We developed an additional dataset to serve as the supplementary information by taking the top 20,000 most popular PyPI packages and prompting LLaMA-2 to generate a list of five questions that each package could help answer given the description. After removing duplicate responses, this resulted in 65,000 statements in the form “Package [x] could answer questions about [y]”. These 65,000 statements were stored in a vector database, enabling efficient retrieval of semantically similar statements. When a model was asked to recommend packages given a code generation prompt or Stack Overflow question, the vector database is first queried from within the code to return the top 5 most semantically similar statements. These statements are appended to the prompt to give the model additional information containing established valid packages to assist in generating non-hallucinated responses.

Self-Refinement. Drawing on insights from our findings in RQ3 (see Section 5.3), which revealed that LLMs often exhibit proficiency in identifying their own package hallucinations, we implemented a self-refinement method. Following the generation of package names, the model is queried regarding the validity of these packages. If the model indicates that the packages are invalid, the response is regenerated with a specific instruction to not use the invalid package. This regeneration process is allowed to iterate up to five times, acknowledging that many package hallucinations are persistent, as demonstrated in RQ3, and may be generated repeatedly. It is possible during this test that a valid package is misclassified as a hallucination by the model, although because this is an iterative process, the success rate should outweigh the false positive rate over the course of testing, resulting in a net decrease in hallucination rate.

Fine-tuning. For our next method, we fine-tuned the models using the code/package list (Heuristic 1) and prompt/package list (Heuristic 2) pairs that were generated during our initial

experiments (Section 5.1). All hallucinations were filtered out and the models were re-trained using the remaining valid responses (560,000 samples). As fine-tuning affects the underlying model weights, we also need to make sure the fine-tuned model retains the ability to produce functional and effective code, which we will test by comparing the *code quality* of the original models and the fine-tuned ones using the well-known *HumanEval benchmark* [6]. The HumalEval benchmark is a set of pre-defined prompts and test cases, and the *final score* is a percentage of problems for which the model generated code is both syntactically correct and passes the test cases, reflecting the model’s ability to produce functional and accurate code.

We implemented these mitigation techniques using the DeepSeek Coder 6.7B and CodeLlama 7B models. These models were selected because they represent two distinct classes of foundational models, with these specific parameter sizes reflecting diverse performance levels: DeepSeek being among the best-performing, and CodeLlama among the worst-performing during our initial experiments (see Section 5.1). Both models were tested using each of the above methods individually and then using all three methods in an *ensemble* configuration.

Table 4: Performance of the mitigation techniques.

	DeepSeek	CodeLlama
Baseline (No Mitigations)	16.14%	26.28%
Retrieval Augmented Generation (RAG)	12.24%	13.40%
Self-Refinement	13.04%	25.51%
Fine-tuning	2.66%	10.27%
Ensemble	2.40%	9.32%

Results. Overall, all the mitigation strategies we implemented resulted in a reduced rate of package hallucination, with **RAG and Supervised Fine-Tuning proving particularly effective** (see Table 4). Fine-tuning proved to significantly improve the results, especially for the DeepSeek model, where hallucinations were reduced by 83%, achieving a total rate of just 2.66%, which is a lower rate than any of the ChatGPT models (observed during RQ1). Self-refinement feedback was also much more effective for the DeepSeek model (19% reduction) compared to the CodeLlama model (3% reduction). This aligns with our results in RQ3, where the DeepSeek model was proficient at detecting hallucinations, while the CodeLlama model had a strong bias towards labeling packages as valid, which limited its ability to reliably detect errors. The ensemble method of combining all mitigation strategies further improved the results, reducing hallucination rates by 85% and 64% from their baseline levels for DeepSeek and CodeLlama, respectively.

The code quality of the fine-tuned models, as measured using the HumanEval benchmark tests, did decrease significantly, -26.1% and -3.1% for DeepSeek and CodeLlama re-

spectively, in exchange for substantial improvements in package hallucination rate. Although code quality was negatively affected, fine-tuned scores are still at levels comparable to other high-performing models such as Mistral 7B (26.1%), Llama 65B (23.1%), and Llama 2 7B (12.8%).

In summary, our results demonstrate that while all tested mitigation strategies effectively reduce package hallucinations, fine-tuning comes at the cost of diminished code quality. Further research is needed to develop fine-tuning methods that minimize hallucinations without compromising quality. In the meantime, RAG and self-refinement offer promising alternatives.

Table 5: Analysis of code quality after fine-tuning.

	DeepSeek	CodeLlama
Original Model pass@1	51.4%	19.6%
Fine-tuned pass@1	25.3%	16.4%

7 Discussion and Conclusion

Since the writing of this paper, several more advanced models have emerged for code generation. These newer models may offer improved performance and different hallucination tendencies and characteristics, which we may have been unable to capture in our study. The study also includes fewer commercial models due to funding constraints, meaning that the findings may not fully represent the performance and hallucination tendencies of the latest commercial LLMs.

In terms of future work, understanding the precise underlying causes of package hallucinations is still an open question. This includes exploring the architecture and components of LLMs that can contribute to these errors, examining the adequacy of tokenizers, and assessing how the composition of the training data and pre-processing impact hallucination tendencies. Identifying and mitigating these underlying issues could lead to more robust and reliable code generation models. Future work could also focus on developing and testing more sophisticated mitigation strategies tailored specifically for code generation tasks. This could involve advanced techniques in prompt engineering, the use of complex knowledge graphs, the refinement of loss functions, and the exploration of new fine-tuning methods. Integrating real-time feedback mechanisms to dynamically adjust model output could further reduce hallucination rates. Understanding how package hallucinations are systemic and persistent at the token level remains crucial.

In conclusion, we systematically studied package hallucinations in code-generating LLMs, including both commercial and open-source models. Our comprehensive analysis revealed that 19.7% of the generated packages are fictitious, posing a critical threat to software security through package

confusion attacks. We identified key behavioral patterns and characterized hallucinated packages, proposing effective mitigation strategies. Our findings underscore the importance of addressing package hallucinations to enhance the reliability and security of AI-assisted software development.

Ethics Considerations

We have disclosed our research to model providers including OpenAI, Meta, DeepSeek, and Mistral AI. We received responses from all providers that our research will be taken into consideration for future models.

Our research highlights the feasibility of a new attack vector that can be used to carry out package confusion attacks by exploiting the package hallucinations generated by code-generating LLMs. Hallucinations are a well-studied limitation of generative AI models, including code-generating LLMs, which even current state-of-the-art techniques cannot completely eliminate. Our goal in this research is to get a better understanding of this phenomenon of package hallucinations in code-generating LLMs. We hope that these findings can help secure future models so that users are better protected against package confusion threats enabled due to package hallucinations.

One course of action that we chose not to pursue for ethical reasons was publishing actual packages using hallucinated package names to PyPI (with no actual code). There would be scientific value in demonstrating the validity of exploiting package hallucinations on an open-source repository, but we felt that doing so would have been misleading, undermined the integrity of the repository, wasted resources, and could be interpreted as violating the terms of service.

Another consideration for making our data public is that half of our prompt dataset was compiled using user created data from StackOverflow. This data consisted of carefully scraped questions and the only data obtained was the question itself; we did not collect any user information or even the answers to the question. This approach aligns with privacy best practices and adheres to data minimization principles, which dictate collecting only the data necessary for our research objectives. Furthermore, the process of scraping was designed to comply with StackOverflow's terms of service and data use policies, ensuring that we maintained legal and ethical integrity throughout the data collection phase. This careful consideration in data handling not only safeguards user privacy but also reinforces the ethical standards we uphold in making our dataset publicly available.

No human subjects were involved in our research and all experiments were conducted in controlled settings with no impact on external persons or entities.

Open Science

We are committed to the open science policy and will make all source code, datasets, and generated code publicly available. The only exception is we will not release our master list of hallucinated package names or the detailed results of our individual tests to the public. Such information could be misused by malicious actors to execute a package hallucination attack. The master list and full results from each of the 30 tests presented in Section 5.1 will be shared responsibly by request to verified researchers.

Specific items that will be made available:

1. All code and data necessary to recreate our primary experiments to detect hallucinated packages.
 - Given a model, a single script will generate all code and packages then evaluate the output for hallucinations and save the results.
 - Both the LLM and Stack Overflow datasets.
 - An environment.yml file that details the packages and dependencies needed to run the code.
2. All code and data to reproduce each figure in the paper.
3. All code and data to reproduce the mitigation tests in the paper.
 - Two fine-tuned models trained on question/answer responses containing valid package names.
 - RAG database along with the code and data used to create it.
 - Code to run all 4 mitigation tests.
4. All data and code can be found at: <https://zenodo.org/records/14676377> or <https://github.com/Spracks/PackageHallucination>.

Acknowledgements

This research was partially supported by the University of Texas at San Antonio (UTSA) Faculty Development Leave (FDL) program and by the National Science Foundation (NSF) under award number 2231002.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv:2303.08774*, 2023.
- [2] Renat Aksitov, Chung-Ching Chang, David Reitter, Siamak Shakeri, and Yunhsuan Sung. Characterizing attribution and fluency tradeoffs for retrieval-augmented large language models. *arXiv:2302.05578*, 2023.

- [3] Anthropic. The Claude 3 Model Family: Opus, Sonnet, Haiku. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf, 2023. [Online; accessed 15-May-2024].
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *NIPS*, 33:1877–1901, 2020.
- [5] Haw-Shiuan Chang and Andrew McCallum. Softmax bottleneck makes language models unable to represent multi-mode word distributions. In *ACL*, 2022.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
- [7] Xiuying Chen, Mingzhe Li, Xin Gao, and Xiangliang Zhang. Towards improving faithfulness in abstractive summarization. *NIPS*, 2022.
- [8] Nouha Dziri, Andrea Madotto, Osmar Zaiane, and Avishek Joey Bose. Neural path hunter: Reducing hallucination in dialogue systems via path grounding. In *EMNLP*, 2021.
- [9] Python Software Foundation. Python Package Index - PyPI. <https://pypi.org>, 2024. [Online; accessed 15-May-2024].
- [10] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *ICLR*, 2023.
- [11] Isabel O Gallegos, Ryan A Rossi, Joe Barrow, Md Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nesreen K Ahmed. Bias and fairness in large language models: A survey. *arXiv:2309.00770*, 2023.
- [12] GitHub. Octoverse: The state of open source and rise of AI in 2023. <https://github.blog/2023-11-08-the-state-of-open-source-and-ai>, 2023. [Online; accessed 15-May-2024].
- [13] Google. BigQuery. <https://cloud.google.com/bigquery>, 2024. [Online; accessed 15-May-2024].
- [14] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming—the rise of code intelligence. *arXiv:2401.14196*, 2024.
- [15] Julian Hazell. Spear phishing with large language models. *arXiv:2305.06972*, 2023.
- [16] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *ICLR*, 2020.
- [17] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint:2311.05232*, 2023.
- [18] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv:2311.05232*, 2023.
- [19] Minlie Huang, Xiaoyan Zhu, and Jianfeng Gao. Challenges in building intelligent open-domain dialog systems. *TOIS*, 2020.
- [20] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023.
- [21] Ziwei Ji, Zihan Liu, Nayeon Lee, Tiezheng Yu, Bryan Wilie, Min Zeng, and Pascale Fung. RHO: Reducing hallucination in open-domain dialogues with knowledge grounding. In *ACL*, 2023.
- [22] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. Towards mitigating LLM hallucination via self reflection. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *EMNLP*, 2023.
- [23] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv:2310.06825*, 2023.
- [24] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv:2401.04088*, 2024.
- [25] Daniel Kang, Xuechen Li, Ion Stoica, Carlos Guestrin, Matei Zaharia, and Tatsunori Hashimoto. Exploiting programmatic behavior of llms: Dual-use through standard security attacks. *arXiv preprint:2302.05733*, 2023.

- [26] Berkay Kaplan and Jingyu Qian. A survey on common threats in npm and pypi registries. In *MLHat*, 2021.
- [27] Reversing Labs. VMConnect supply chain attack continues, evidence points to North Korea. <https://www.reversinglabs.com/blog/vmconnect-supply-chain-campaign-continues>, 2023. [Online; accessed 15-May-2024].
- [28] P. Ladisa, H. Plate, M. Martinez, and O. Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *SP*, 2023.
- [29] Bar Lanyado. Can you trust chatgpt’s package recommendations? <https://lasso-security.webflow.io/blog/ai-package-hallucinations>, 2023. [Online; accessed 15-May-2024].
- [30] Bar Lanyado. Diving Deeper into AI Package Hallucinations. <https://vulcan.io/blog/ai-hallucinations-package-risk>, 2023. [Online; accessed 15-May-2024].
- [31] Vladimir I Levenshtein et al. Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet physics doklady*, 1966.
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *NIPS*, 2020.
- [33] Chenliang Li, Bin Bi, Ming Yan, Wei Wang, and Songfang Huang. Addressing semantic drift in generative question answering with auxiliary extraction. In *ACL-IJCNLP*, 2021.
- [34] Haoran Li, Dadi Guo, Wei Fan, Mingshi Xu, Jie Huang, Fanpu Meng, and Yangqiu Song. Multi-step jailbreaking privacy attacks on chatGPT. In *EMNLP*, 2023.
- [35] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. *NIPS*, 2024.
- [36] Jenny T Liang, Chenyang Yang, and Brad A Myers. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *ICSE*, 2024.
- [37] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *ACL*, 2022.
- [38] Bingbin Liu, Jordan Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Exposing attention glitches with flip-flop language modeling. *NIPS*, 2024.
- [39] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv:2404.00971*, 2024.
- [40] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *NIPS*, 2024.
- [41] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. Jailbreaking chatgpt via prompt engineering: An empirical study. *arXiv:2305.13860*, 2023.
- [42] Yijin Liu, Xianfeng Zeng, Fandong Meng, and Jie Zhou. Instruction position matters in sequence generation with large language models. *arXiv:2308.12097*, 2023.
- [43] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. In *ICLR*, 2024.
- [44] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *NIPS*, 2024.
- [45] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan Thomas McDonald. On faithfulness and factuality in abstractive summarization. In *ACL*, 2020.
- [46] Ida Momennejad, Hosein Hasanbeig, Felipe Vieira Frjeri, Hiteshi Sharma, Nebojsa Jojic, Hamid Palangi, Robert Ness, and Jonathan Larson. Evaluating cognitive maps and planning in large language models with cogeal. *NIPS*, 2024.
- [47] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. Beyond typosquatting: an in-depth look at package confusion. In *USENIX*, 2023.
- [48] npm. <https://npmjs.com>, 2024. [Online; accessed 15-May-2024].
- [49] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber’s knife collection: A review of open source software supply chain attacks. In *DIMVA*, 2020.
- [50] Yasumasa Onoe, Michael Zhang, Eunsol Choi, and Greg Durrett. Entity cloze by date: What LMs know about unseen entities. In *NAACL*, 2022.

- [51] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of github copilot’s code contributions. In *SP*, 2022.
- [52] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do users write more insecure code with ai assistants? In *CCS*, 2023.
- [53] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint:2308.12950*, 2023.
- [54] Sonatype. 9th Annual State of the Software Supply Chain. <https://www.sonatype.com/state-of-the-software-supply-chain/introduction>, 2023. [Online; accessed 15-May-2024].
- [55] Sonatype. Top 8 malicious attacks recently found on PyPI. <https://blog.sonatype.com/top-8-malicious-attacks-recently-found-on-pypi>, 2023. [Online; accessed 15-May-2024].
- [56] stackoverflow. Stack Overflow. <https://stackoverflow.com/>, 2024. [Online; accessed 15-May-2024].
- [57] Katherine Tian, Eric Mitchell, Huaxiu Yao, Christopher D Manning, and Chelsea Finn. Fine-tuning language models for factuality. In *ICLR*, 2024.
- [58] TIDELIFT. Libraries.io. <https://libraries.io>, 2024. [Online; accessed 15-May-2024].
- [59] SM Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv:2401.01313*, 2024.
- [60] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*, 2023.
- [61] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. Bad snakes: Understanding and improving python package index malware scanning. In *ICSE*, 2023.
- [62] Chaojun Wang and Rico Sennrich. On exposure bias, hallucination and domain shift in neural machine translation. In *ACL*, 2020.
- [63] Guan Wang, Sijie Cheng, Xianyuan Zhan, Xiangang Li, Sen Song, and Yang Liu. Openchat: Advancing open-source language models with mixed-quality data. *arXiv:2309.11235*, 2023.
- [64] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does llm safety training fail? *NIPS*, 2024.
- [65] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Source code is all you need. *arXiv:2312.02120*, 2023.
- [66] Junan Zhang, Kaifeng Huang, Bihuan Chen, Chong Wang, Zhenhao Tian, and Xin Peng. Malicious package detection in npm and pypi using a single model of malicious behavior sequence. *arXiv:2309.02637*, 2023.
- [67] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. Siren’s song in the ai ocean: A survey on hallucination in large language models. *arXiv:2309.01219*, 2023.

A Truncated List of LLM Generated Coding Prompts

- 1) Generate Python code that implements a simple web server that can handle GET and POST requests using the `http.server` module.
 - 2) Generate Python code that imports the AWS SDK for Python and creates an Amazon S3 bucket, an Amazon EC2 instance, and an Amazon RDS database, and sets up user authentication using IAM roles.
 - 3) Generate Python code that implements a simple flock-based file locking mechanism using the ‘with’ syntax, allowing multiple threads to safely access a shared resource while ensuring exclusive access for writing operations.
 - 4) Generate Python code that implements a backport of f-strings to Python versions prior to 3.6 using the ‘`str.format()`’ method.
 - 5) Generate Python code that imports the Selenium library and uses it to automate interactions with a web application, such as navigating to pages, filling out forms, and verifying expected elements are present on the page.
 - 6) Generate Python code that implements a rate limiter for Flask applications using the ‘`limiter`’ library, which provides a simple way to add rate limiting to any Flask endpoint.
- ⋮
- 9810) Generate Python code that imports the PyGlove library and uses it to manipulate various Python objects, such as lists, dictionaries, and strings, by applying operations like reversal, sorting, indexing, slicing, concatenation, and membership testing.

- 9811) Generate Python code that imports the necessary libraries and uses the Fuzzy Self-Tuning PSO algorithm to perform global optimization for a given function.
- 9812) Generate Python code that imports the necessary libraries and sets up a configurable middleware pipeline for making HTTP requests to the Microsoft Graph API using the Core component of the Microsoft Graph Python SDK.
- 9813) Generate Python code that imports the necessary CUDA libraries and creates a simple kernel that performs a matrix multiplication using CUDA's GPU acceleration.
- 9814) Generate Python code that imports the threading module and uses it to create threads for monitoring and tracing in an application, using the OpenCensus API to collect metrics and trace data.

B System Messages and Prompts

Figures 10 to 12 below show the system messages and prompts that are sent to each model to generate the code samples and package lists.

System Message

You are a coding assistant that generates [Python/JavaScript] code. Provide only the code and add additional explanatory text only when absolutely necessary. If no code is required to answer the question, simply reply 'None'.

Prompt

LLM Generated prompt from Appendix A or Stack Overflow question.

Figure 10: Code generation phase.

C Model Parameters and Testing Environment

Model testing was conducted in two distinct computing environments - a Debian environment with 40 nodes, each equipped with 40 CPU cores, 1TB of RAM, and NVIDIA A100 or V100 GPUs and a Ubuntu system with 80 CPU cores, 750 GB of RAM, and 3 NVIDIA RTX 6000 GPUs. Table 6 shows the model parameters used during our RQ1 tests.

D Python vs. JavaScript Hallucination

The linear relationship demonstrating a model's propensity to hallucinate across both Python and JavaScript (as described in Section 5.1) is shown in Figure 14.

System Message

You are a coding assistant that determines [Python/JavaScript] packages necessary to execute code. Respond with only a list of [Python/JavaScript] packages, separated by commas and no additional text or formatting. If there is no code provided, respond 'None', otherwise the response must begin with the name of a [Python/JavaScript] package.

Prompt

[Python/JavaScript] packages are required to run this code: [Code sample generated from code generation phase].

Figure 11: Package generation phase - Heuristic 1.

System Message

You are a coding assistant that recommends [Python/JavaScript] packages that would be helpful to solve given problems. Respond with only a list of [Python/JavaScript] packages, separated by commas and no additional text or formatting. The response must begin with the name of a [Python/JavaScript] package.

Prompt

Which [Python/JavaScript] packages would be useful in solving the following coding problem: [Original LLM generated prompt or Stack Overflow question].

Figure 12: Package generation phase - Heuristic 2.

E Complete Results for Python and JavaScript

Table 7 and Table 8 shows complete results for package hallucination experiments observed across all tested models for Python and JavaScript.

F Additional Background Information

F.1 Hallucinations

Hallucinations can also be categorized based on whether they can be directly verified from the source content; if so, it is termed an *intrinsic hallucination*, otherwise, it is considered an *extrinsic hallucination* [20]. The structured nature of code leads to intrinsic hallucinations that are directly traceable to syntactic errors, while extrinsic hallucinations arise from complex interactions or gaps in the model's training data [39]. Code generation hallucinations manifest in several ways,

System Message

You are a coding assistant that assists users in creating simple prompts that will be used to generate [Python/JavaScript] code. No code should be used in the response.

Prompt

Your answer must begin with 'Generate [Python/JavaScript] code that' and must not be longer than one sentence. Do not include extra text or formatting (i.e. do not start with 'Sure! Here's a prompt...'). Write a prompt that would generate [Python/JavaScript] code to accomplish the same tasks as the following package description: [package description from PyPI].

Figure 13: Coding prompt generation.

Table 6: An overview of the model parameters.

Parameter	Value
Temperature (Code Generation)	0.7
Temperature (Package Prompts)	0.01
Top- p	0.9
Top- k	20
Repetition Penalty	1
Max tokens (Code Generation)	2048
Max tokens (Package Prompts)	64
Typical- p	1
Epsilon Cutoff	0
Eta Cutoff	0
Diversity Penalty	0

including functional bugs that impair the intended operation, code that performs the wrong task, dead code that never gets executed, and, perhaps most critically, security vulnerabilities that can be exploited.

The persistent issue of hallucinations in LLMs has spurred extensive research into various mitigation strategies for standard natural language generation tasks, broadly categorized into *prompt engineering* and *model architecture enhancements* [59]. Prompt engineering techniques such as Retrieval Augmented Generation (RAG) [32] and self-refinement methods [44] aim to refine the input provided to the model to produce more accurate outputs. Alternatively, developing more robust models involves approaches such as supervised fine-tuning [57], inference time intervention [35], and incorporating knowledge graphs [21] to improve model understanding and reduce errors/hallucinations in the model output.

The manipulation of the *temperature* parameter within each model has also been shown to significantly influence the

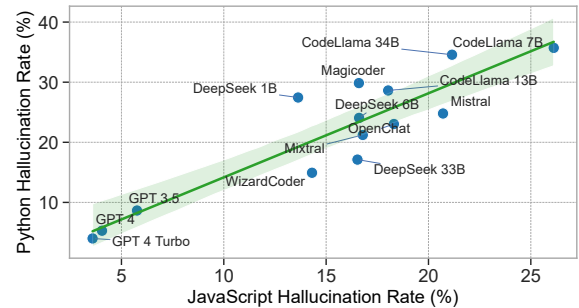


Figure 14: Python vs. JavaScript hallucination rates.

prevalence of hallucinations in LLM output [2]. The temperature parameter modulates the probability distribution over potential output tokens [46]. Lower temperatures produce less random, more predictable outputs, while higher temperatures increase the likelihood of sampling low-frequency tokens, raising the risk of hallucinations [8].

G Packages and Modules Explanation

Here is a quick primer on the purpose of packages and modules in coding languages and how that relates to the topic of package hallucinations. To enable code modularity and reusability, interpreted programming languages such as Python and JavaScript allow for entities called *modules*. A module is a chunk of code, often in an external file, that performs a specific task or function. By encapsulating related code into modules, developers can organize their programs more efficiently and make them easier to maintain and share. A *package* is a collection of related modules that work together to provide certain functionality. To use a particular module in their source code, a developer must install the appropriate package into its development environment by first downloading the package from an online package manager or repository such as PyPI or npm, if it is not locally available, and then import the desired module into the code using appropriate import functions, e.g., `import` in Python or `require` in JavaScript.

These module names do not necessarily need to match the package names and the namespace for modules is not protected, i.e., different packages may include modules of the same name. This discrepancy poses a significant challenge for detecting package dependencies from the raw Python and JavaScript code, as the `import/require` statements that are typically included in code samples for importing modules do not have a unique mapping to package names. Thus, it would not be accurate to simply parse the generated code for `import` or `require` statements, as those statements refer to module names and not package names.

Table 7: Hallucination Percentages for all models tested using Python code.

Model	Total Hallucination	LLM Generated Prompts	Stack Overflow Prompts	“pip install”
GPT-4 Turbo	3.59% (2,739/76,313)	3.29% (1,518/46,204)	4.07% (1,169/28,728)	3.77% (52/1,381)
GPT-4	4.05% (2,969/73,396)	3.83% (1,741/45,403)	4.45% (1,046/23,487)	4.04% 182/4,506
GPT-3.5 Turbo	5.76% (4,387/76,123)	5.98% (2,495/41,7)	5.50% (1,868/33,948)	5.63% (24/426)
DeepSeek 1B	13.63% (12,481/91,543)	11.07% (5,847/52,806)	16.39% (5,901/36,007)	26.85% (733/2,730)
DeepSeek 33B	16.53% (7,071/42,788)	13.85% (3,623/26,167)	25.47% (3,033/11,906)	8.80% (415/4,715)
WizardCoder 33B	14.31% (4,909/34,300)	9.79% (1,579/16,125)	21.40% (2,852/13,329)	9.84% (477/4,846)
DeepSeek 6B	16.61% (16,526/99,505)	14.01% (9,240/65,957)	23.56% (6,792/28,828)	10.47% (494/4,720)
OpenChat 7B	18.31% (16,932/92,452)	17.39% (9,582/55,092)	19.98% (6,454/32,307)	17.73% (896/5,053)
CodeLlama 13B	18.03% (12,404/68,809)	15.21% (6,450/42,410)	22.76% (5,752/25,273)	17.94% (202/1,126)
Mixtral 8x7B	16.79% (7,753/46,166)	13.12% (2,749/20,951)	20.92% (4,068/19,949)	16.23% (936/5,766)
MagiCoder 7B	16.60% (20,258/122,057)	15.76% (11,994/76,096)	18.48% (7,621/41,248)	13.64% (643/4,713)
CodeLlama 34B	21.15% (24,905/117,777)	15.22% (9,495/62,366)	28.56% (14,891/52,135)	15.84% (519/3,276)
Mistral 7B	20.71% (7,959/38,437)	14.47% (2,808/19,412)	30.69% (3,922/12,778)	19.67% (1,229/6,247)
WizardCoder 7B - Python	20.69% (11,408/55,131)	16.80% (4,698/27,962)	26.73% (6,112/22,867)	13.90% (598/4,302)
CodeLlama 34B - Python	20.97% (12,128/57,833)	19.01% (5,913/31,112)	23.39% (6,208/26,540)	3.87% (7/181)
CodeLlama 7B	26.12% (27,814/106,487)	21.51% (12,961/60,261)	32.53% (14,671/45,099)	16.15% (182/1,127)

Table 8: Hallucination percentages for all models tested using JavaScript code.

Model	Total Hallucination	LLM Generated Prompts	Stack Overflow Prompts	“npm install”
GPT-4 Turbo	4.00% (2,101/52,484)	2.57% (735/28,545)	5.58% (1283/23,009)	8.92% (83/930)
GPT-4	5.29% (2,911/55,021)	3.78% (1,116/29,534)	3.86% (1,672/23,416)	5.94% 123/2,071
GPT-3.5 Turbo	8.65% (4,576/52,890)	6.92% (1,930/27,909)	10.46% (2,579/24,662)	21.00% (67/319)
DeepSeek 1B	27.45% (29,305/106,755)	23.96% (14,300/59,681)	31.87% (14,975/46,988)	34.88% (30/86)
DeepSeek 33B	17.12% (10,505/61,373)	13.28% (5,472/41,209)	25.65% (4,940/19,260)	10.29% (93/904)
WizardCoder 33B	14.93% (3,876/25,969)	7.83% (1,038/13,256)	23.31% (2,772/11,894)	8.06% (66/819)
DeepSeek 6B	24.06% (25,178/104,628)	17.36% (9,595/55,255)	31.82% (15,493/48,693)	13.24% (90/680)
OpenChat 7B	23.04% (24,863/107,903)	18.34% (10,275/56,039)	28.18% (14,557/51,657)	14.98% (31/207)
CodeLlama 13B	28.62% (11,984/41,866)	19.10% (3,774/19,757)	37.15% (8,200/22,071)	26.32% (10/38)
Mixtral 8x7B	21.22% (9,429/44,435)	14.83% (2,882/19,436)	27.98% (6,257/22,362)	11.00% (290/2,637)
MagiCoder 7B	29.85% (40,085/134,276)	26.27% (20,703/78,817)	35.10% (19,301/54,982)	16.98% (81/477)
CodeLlama 34B	34.57% (38,607/111,668)	25.18% (13,090/51,995)	42.77% (25,489/59,590)	33.73% (28/83)
Mistral 7B	24.79% (10,505/42,381)	20.60% (4,961/24,083)	34.59% (5,252/15,183)	9.37% (292/3,115)
CodeLlama 7B	35.71% (33,877/94,876)	27.32% (12,103/44,298)	43.07% (21,751/50,507)	32.39% (23/71)